Runs 4–9: Looking for the Gap to GLASS

October 28, 2026

Each section answers four questions: why we ran it, what we changed, what came out, and what we learned. Numbers are test micro-F1 ($\times 100$), mean of 10 seeds. In the figures, yellow marks what is new in that run.

Run 4: normalizing the clique, and the hub graph

- **Why:** the advisor said the clique (an edge between every two nodes of a subgraph) pulls all its nodes to one vector, which is close to just averaging them.
- **What we changed:** (1) clique messages divided by $\sqrt{d_i d_j}$, as in GCN, where d is a node's number of clique neighbors; (2) the same, with each clique edge also weighted by 1/(subgraph size); (3) a hub graph: one extra node per subgraph, and links between extra nodes of subgraphs that share nodes or edges (Figure 1).
- **Result:** (1) and (2) moved every dataset by less than one std. The hub graph gave our best cut_ratio at the time (81.6), but no gain elsewhere. Error did not grow with subgraph size.
- **Learned:** how the clique is weighted is not what holds us back.

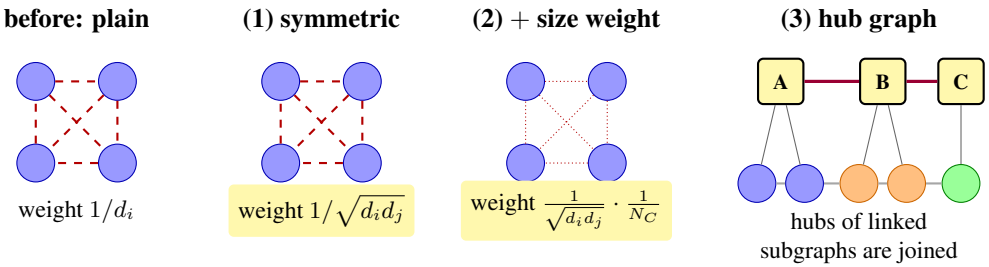


Figure 1: Run 4. Red dashed: clique edges. (1)–(2) change only how much each clique message counts; thinner means it counts less. (3) adds one hub per subgraph (yellow) and purple links between hubs of subgraphs that share nodes or are joined by graph edges.

Check: the GLASS paper

- **Why:** to see what GLASS does that we do not.
- **What we found:** GLASS also reports a plain GNN, with no marks telling the model which nodes are in the subgraph (Figure 2). It gets 59.7 on hpo_metab and 66.8 on hpo_neuro, about 4 points above our best at the time.

- **Learned:** a simple model beats us on the real datasets, so the gap is not about how we mark subgraphs. The question became: what is different in our setup?

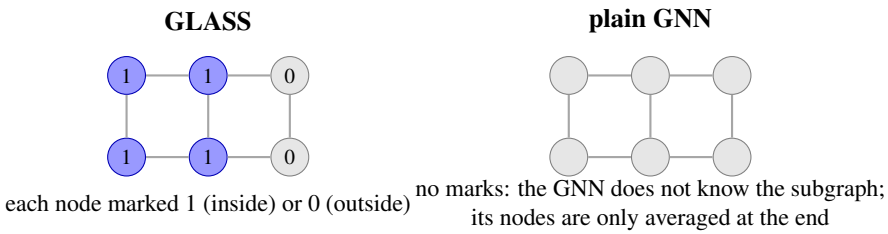


Figure 2: GLASS and its plain GNN, on a subgraph of four nodes. The plain GNN is the simplest possible model, and it still beats us on hpo.

Run 5: node copies and split edges

- **Why:** on density and hpo, many nodes are in more than one subgraph, so one node vector has to serve several subgraphs.
- **What we changed:** (1) copies: each node gets a separate copy for each subgraph it is in; (2) split edges: graph edges inside a subgraph and edges leaving it get separate weights (Figure 3); (3) input vectors started from SubGNN's pretrained ones.
- **Result:** copies lifted density from 78.8 to 92.4, but lowered cut_ratio and component. Split edges gave our best hpo (metab 59.7, neuro 64.1) but broke density (34.4). SubGNN's vectors gave nothing.
- **Learned:** each idea fixes some datasets and breaks others.

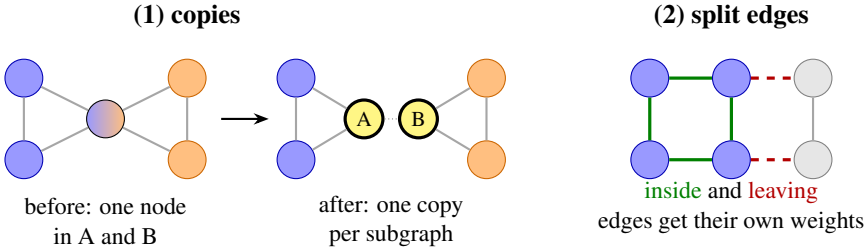


Figure 3: Run 5. (1) The half-blue, half-orange node is in subgraphs A and B; it is replaced by two copies (yellow), each linked only to its own subgraph. (2) Graph edges are split into two types by whether they stay inside a subgraph.

Run 6: copies plus edge counts

- **Why:** in run 5, copies fixed density but lowered cut_ratio (66.0) and component (67.6). Cut_ratio is about how many edges leave the subgraph, and a copy has no direct way to see that. So we tried to hand it this number.
- **What we changed:** we kept run 5's copies model, and gave each copy two numbers as extra input: how many of its graph edges stay inside its subgraph, and how many leave it (Figure 4).

- **Result:** component recovered (67.6 → 98.0) and density rose to 94.4, our best and above GLASS (93.0). Cut_ratio did not move (66.0 → 66.8). The real datasets stayed below our best.
- **Learned:** the model was given the exact numbers cut_ratio depends on, and still did not use them. This made us suspect the training, which run 7 checks.

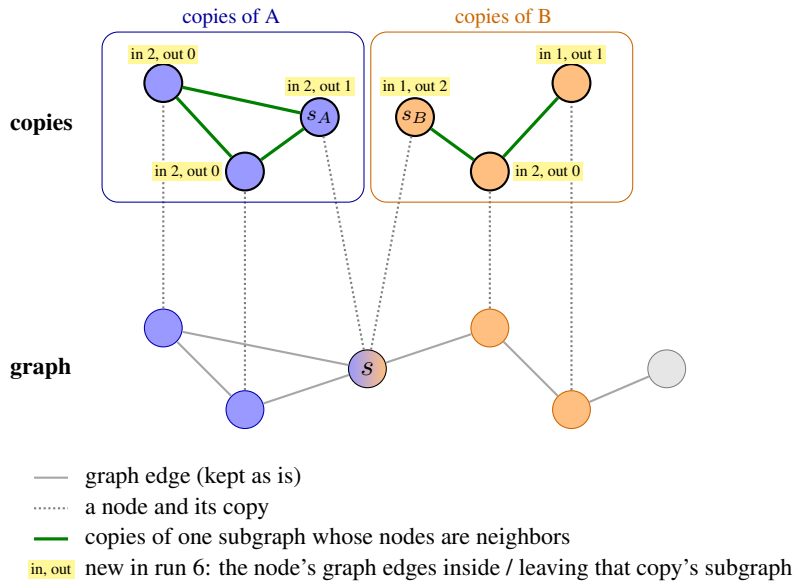


Figure 4: Run 6, the whole graph. Node s is in subgraphs A (blue) and B (orange). Every (node, subgraph) pair gets its own copy, so s gets two: s_A and s_B. The original graph stays, each node is linked to its copies, and copies of one subgraph are linked when their nodes are neighbors in the graph. New in run 6 are the yellow counts. They are counted on the original graph from the copy's subgraph's side, so the same node s gets (2, 1) for A and (1, 2) for B.

Run 7: is it the model or the training?

- **Why:** to find out whether our model cannot express the answer, or our training cannot find it.
- **What we changed:** (1) the subgraph's exact counts (size, inside edges, outside edges) fed straight to the final classifier; (2) GLASS's plain GNN rebuilt in our code (Figure 5).
- **Result:** (1) cut_ratio 68.4, and the model fit only 85% of its own training subgraphs. (2) ppi_bp 62.8 (as good as GLASS), but hpo_metab 46.6 (13 below GLASS).
- **Learned:** the model has what it needs but does not learn it, so the problem is our training.

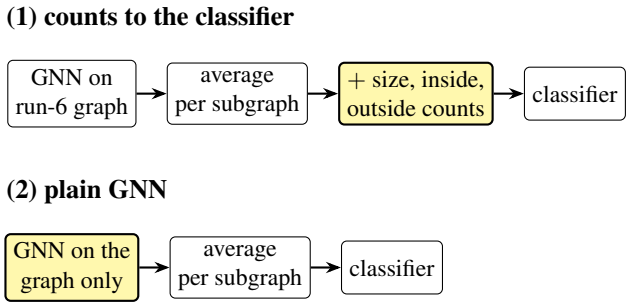


Figure 5: Run 7. (1) The classifier gets the exact counts that define density and cut_ratio, so it only has to learn to read them. (2) No clique, no copies, no hubs: the simplest model, as in GLASS's plain GNN.

Run 8: training like GLASS

- **Why:** GLASS normalizes after each GNN layer and trains on small batches; we did neither.
- **What we changed:** added both to the models of run 7 (Figure 6).
- **Result:** cut_ratio 76.0 (now fits 98% of training), coreness 85.2 (above GLASS). The plain GNN on hpo_metab did not move (46.5).
- **Learned:** normalization fixes part of the synthetic gap, but not hpo_metab.

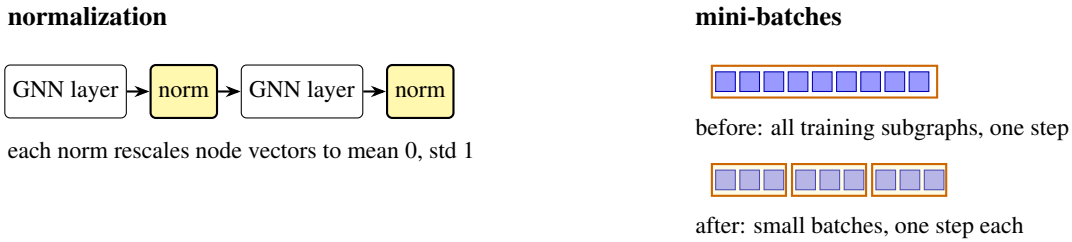


Figure 6: Run 8. Two training changes copied from GLASS; the model itself is the same as in run 7.

Run 9: GLASS's own input vectors

- **Why:** the input was the last difference we could see.
- **What we changed:** started from GLASS's own pretrained node vectors (Figure 7).
- **Result:** plain GNN: hpo_metab 50.0 (still 10 below), hpo_neuro 66.0 (equal to GLASS's plain GNN). Split edges: ppi_bp 64.0, our best ever, above GLASS.
- **Learned:** same input and nearly the same model, and hpo_metab is still far behind. We do not know why.

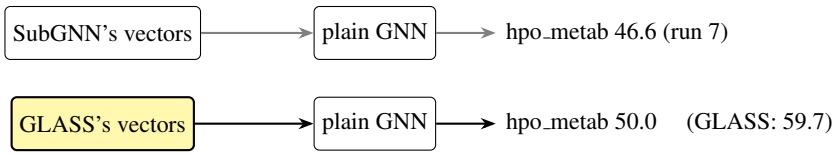


Figure 7: Run 9. Only the starting node vectors change. Gray: the input used before.

Now: running GLASS's own code

- **Why:** to see if GLASS's published numbers can be reached on our server at all.
- **What we are doing:** running the authors' code with their settings on all 8 datasets, for both full GLASS and its plain GNN (Figure 8). (GPEN has no public code.)
- **What it will tell us:** if their plain GNN gets about 60 on hpo_metab, we compare their code with ours step by step. If not, the published number is hard to reach.
- **Already found in their code:** when several epochs tie on validation, it reports the best test score among them; and their "±" is std/$\sqrt{10}$, smaller than the std we report.

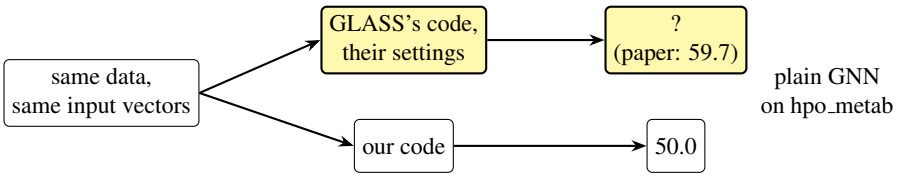


Figure 8: The run going now. If the top line reaches about 60, the gap is in our code and we can find it by comparing the two step by step.

dataset	our best	from	GLASS
ppi_bp	64.0	run 9, split edges + GLASS vectors	61.9
hpo_metab	59.7	run 5, split edges	61.4
hpo_neuro	66.0	run 9, plain GNN + GLASS vectors	68.5
density	94.4	run 6, copies + counts	93.0
cut_ratio	81.6	run 4, hub graph	93.5
coreness	89.1	run 3, transformer readout	84.0
component	100.0	run 7, counts to classifier	100.0

Table 1: Test micro-F1 ($\times 100$), mean over 10 seeds. "Our best" is the best of runs 2–9 on each dataset, from different models. GLASS numbers are from the GLASS paper.